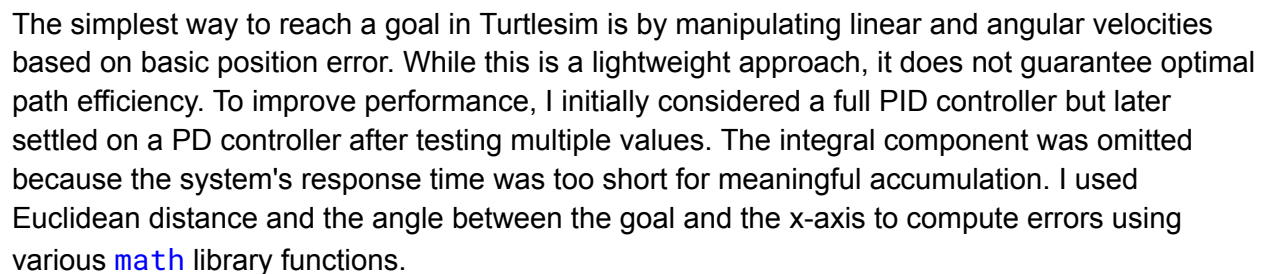


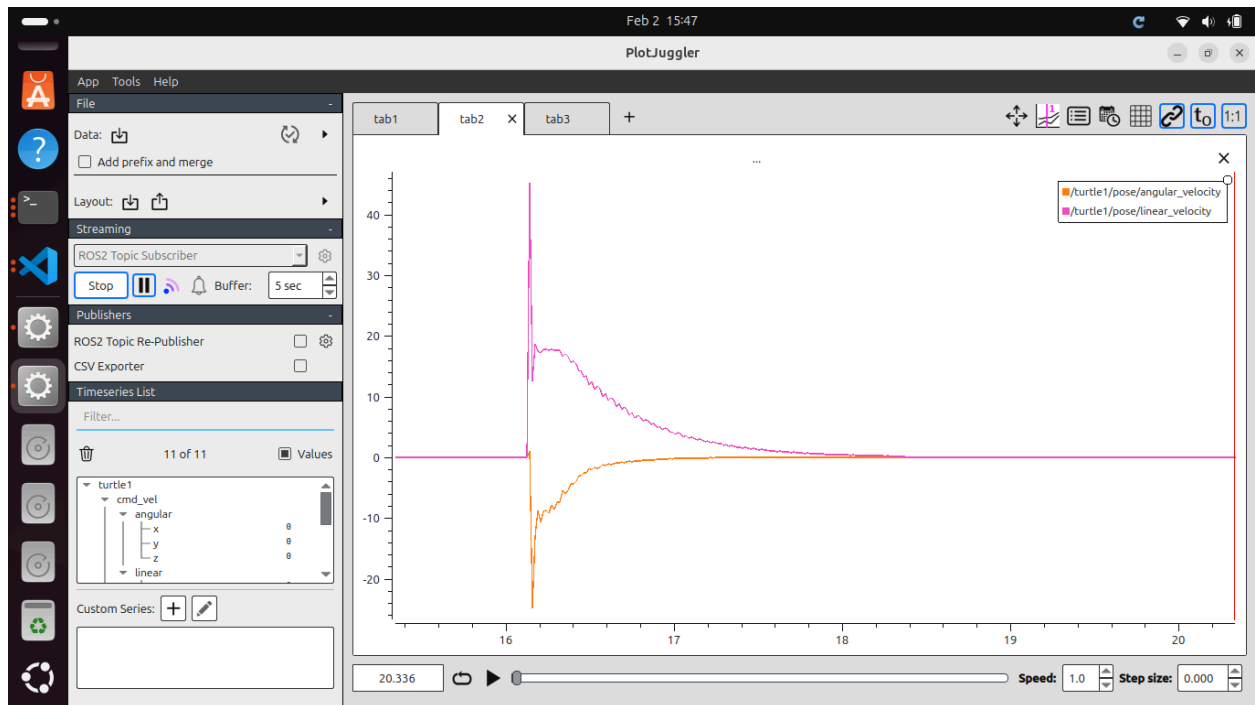
Project Report

My approach for spawning the robot involved using the spawn client to send an asynchronous request to the spawn service. The robot's spawn location was randomly selected within a defined range using the `random` library. The robot subscribes to `/turtle1/cmd_vel`, which receives velocity commands from the `pid_controller_node`. We can either use the default spawned turtle or spawn an additional turtle at a random location using `spawn_node`.

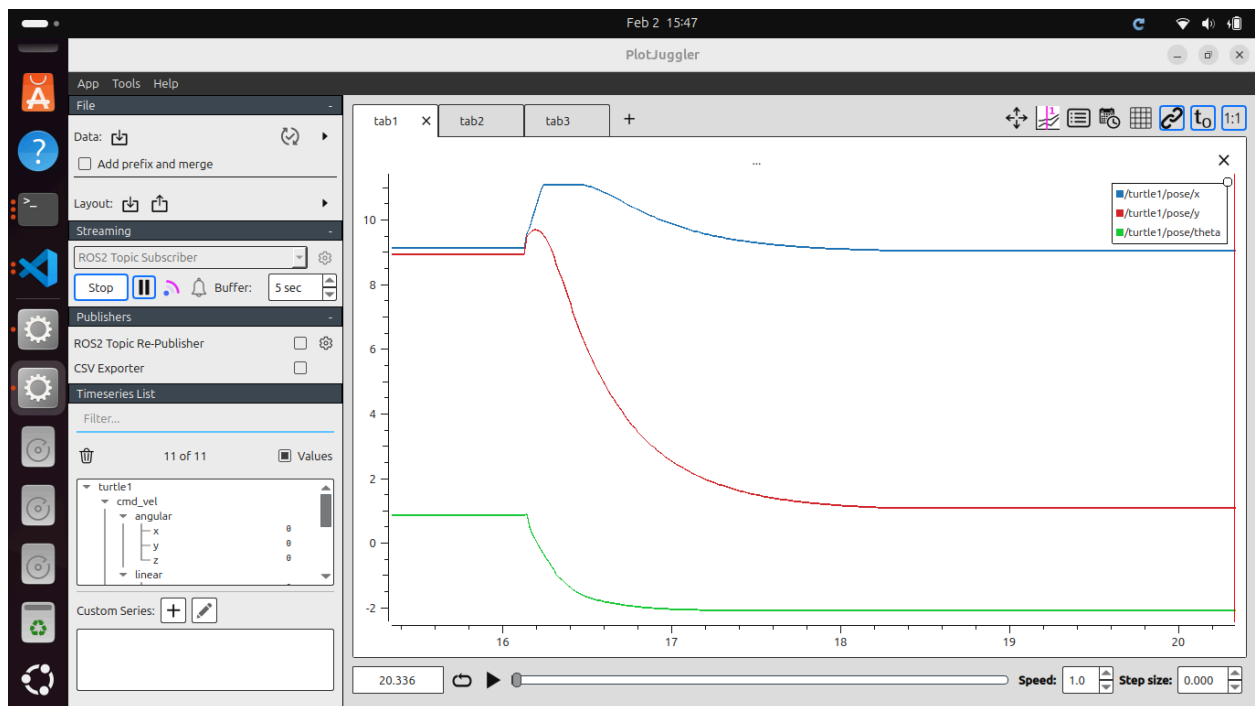


For visualization, I used **PlotJuggler**, which required publishing relevant data, including the turtle's position (`x`, `y`, `theta`), linear and angular errors, and velocity values. I used `std_msgs/Float64` for message types since it could handle all anticipated values. The processed velocity was then published to `/cmd_vel`.

Plot of Linear and angular velocity against time.

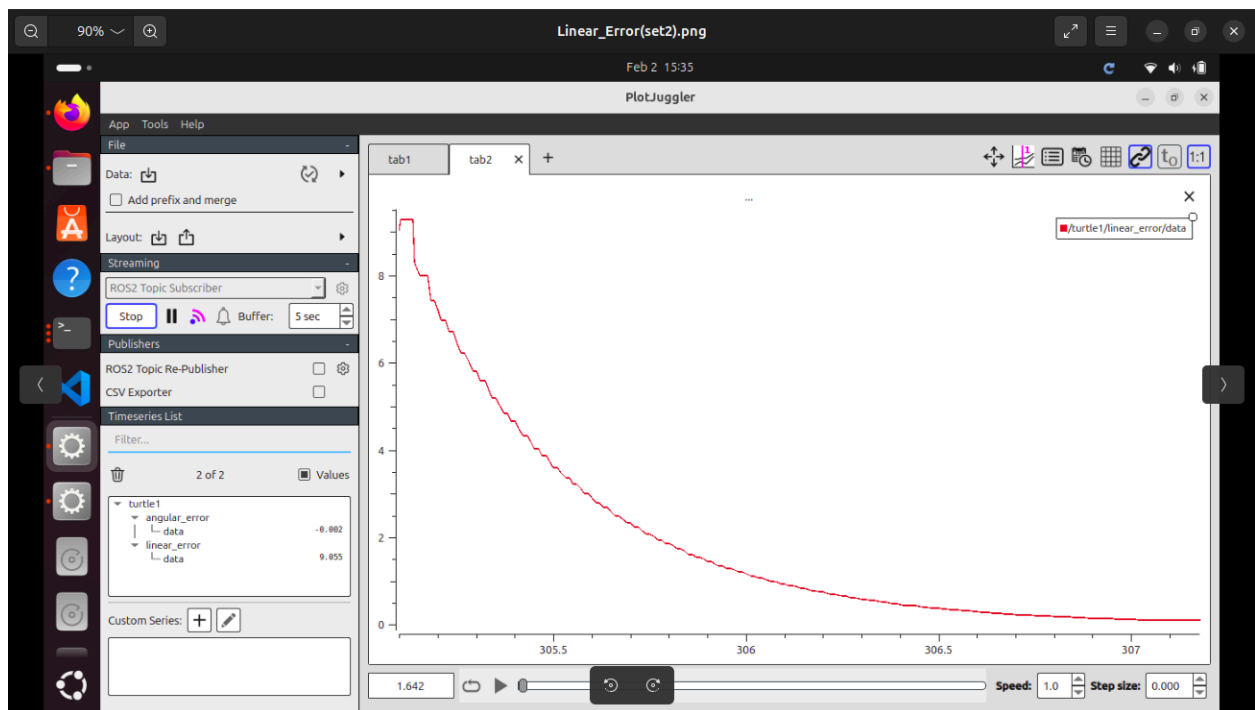
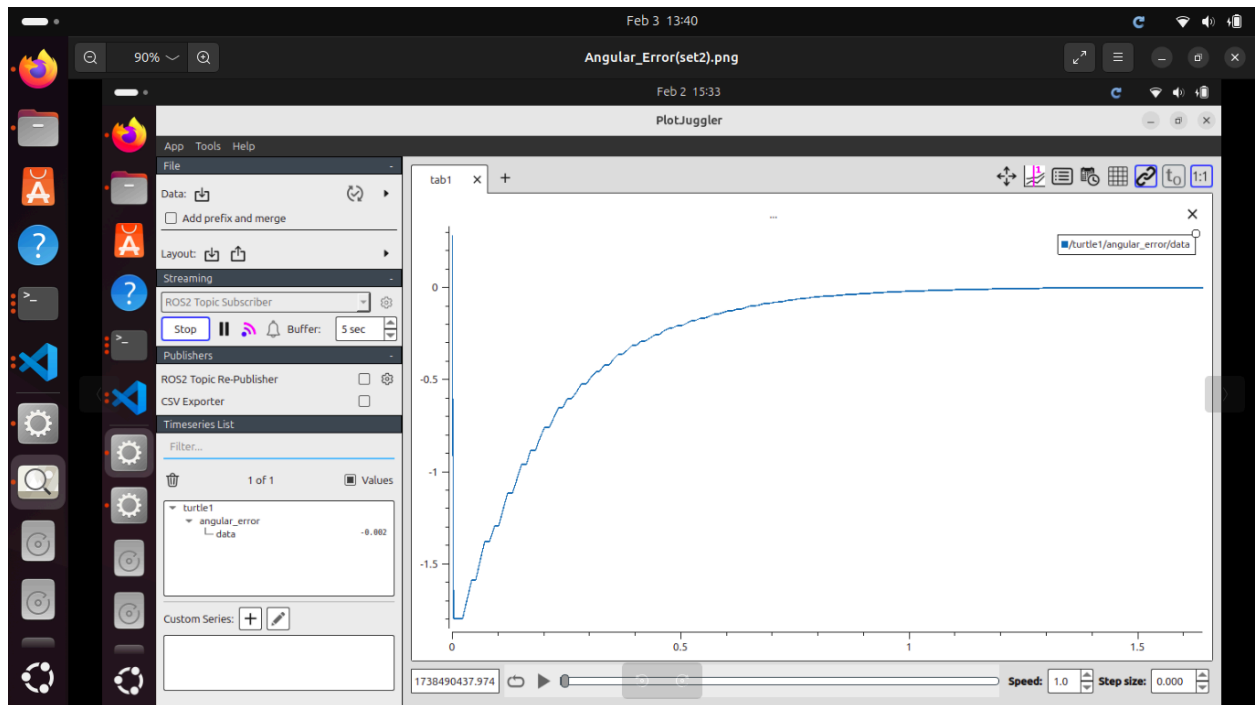


Plot of the position of the turtlebot.

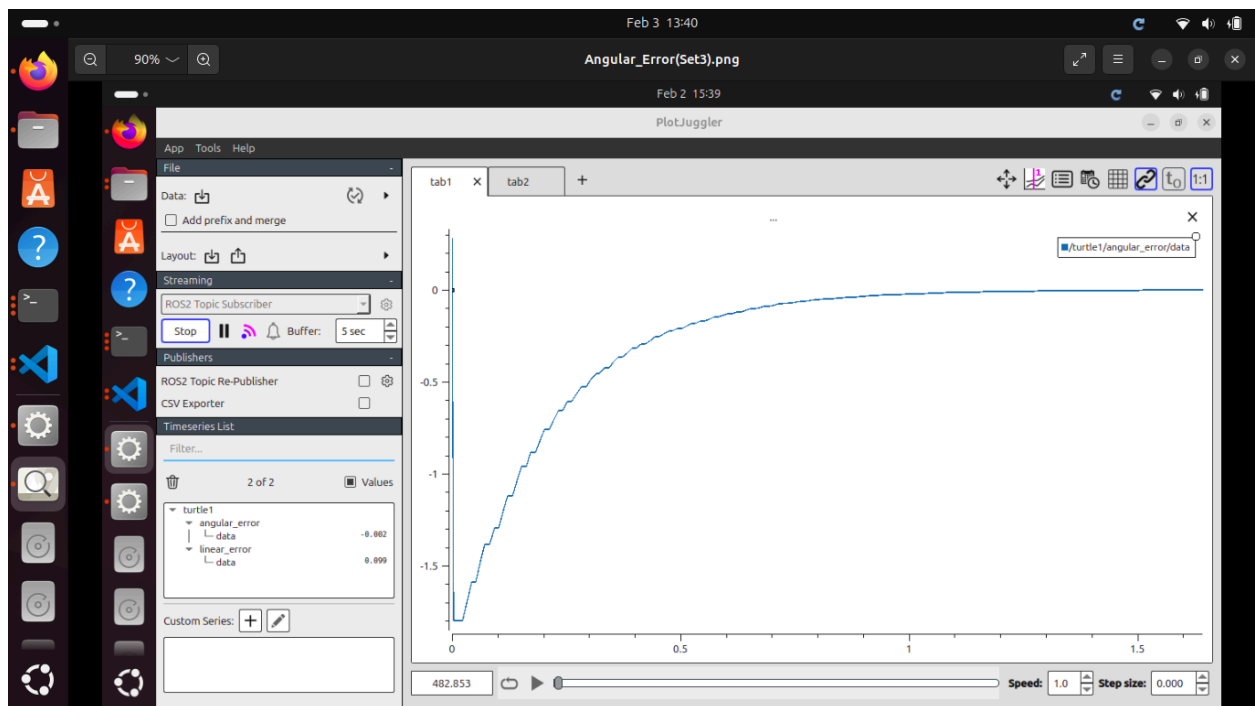
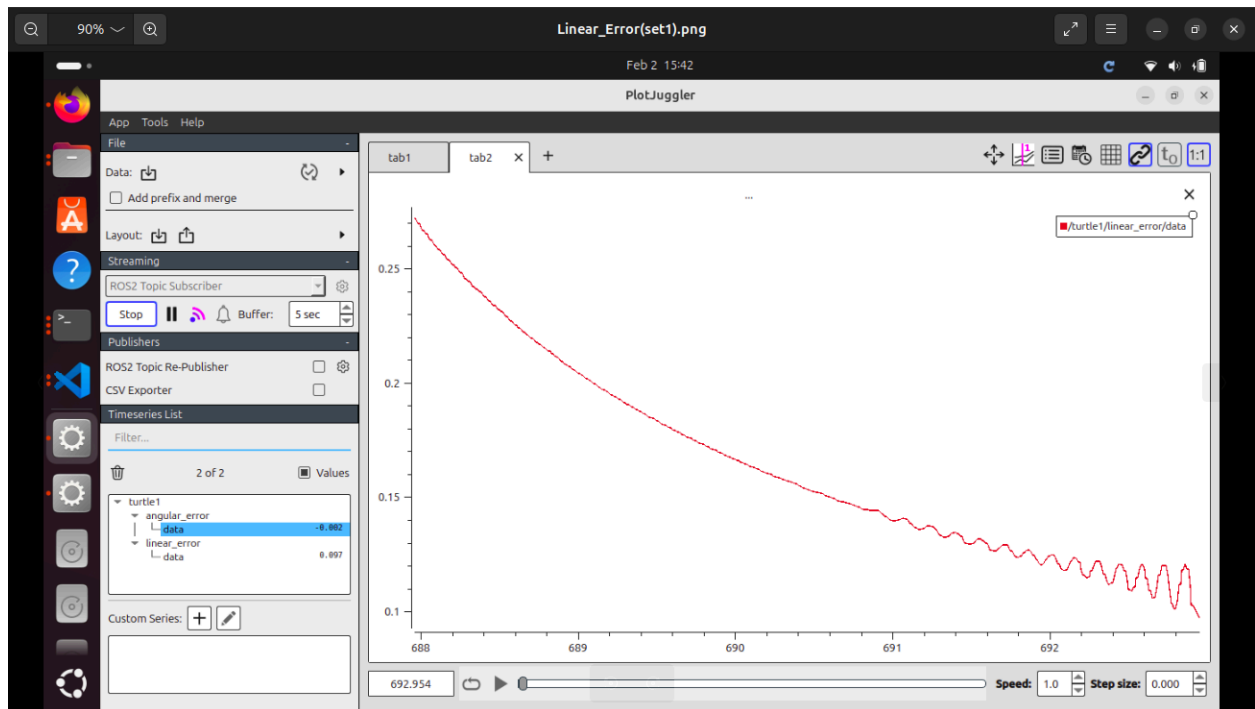


Some gain values resulted in erratic behavior, which required tuning.

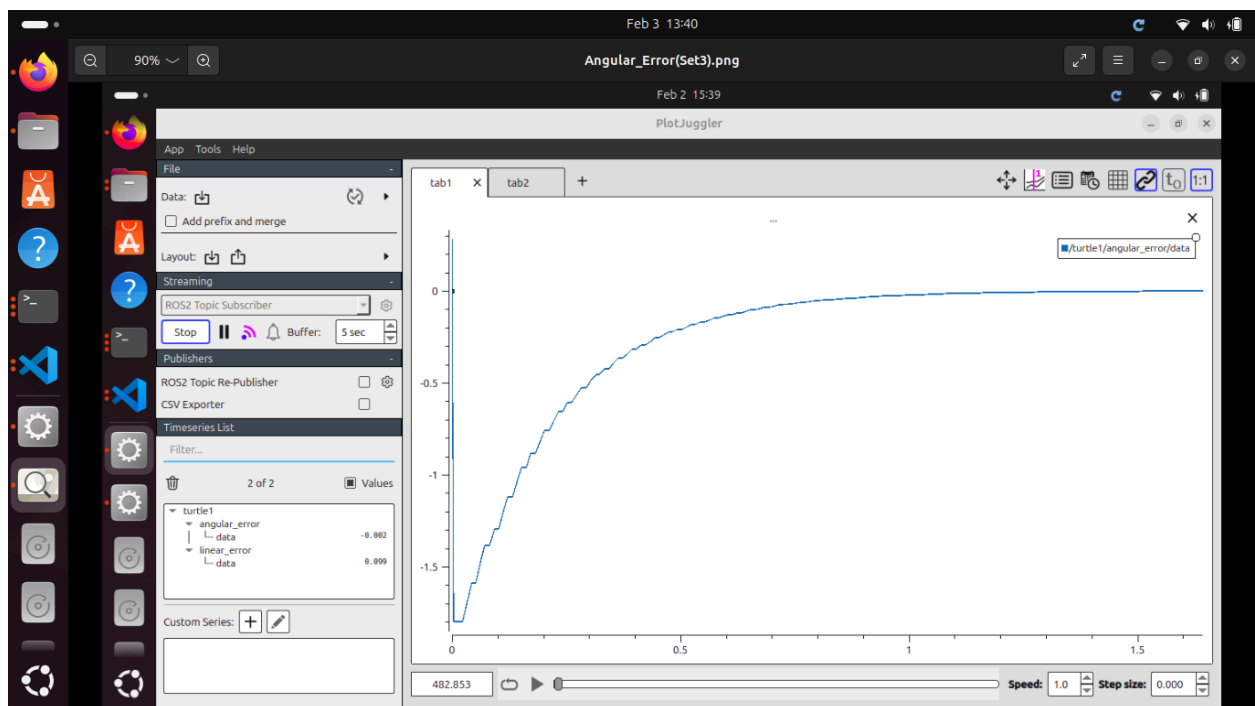
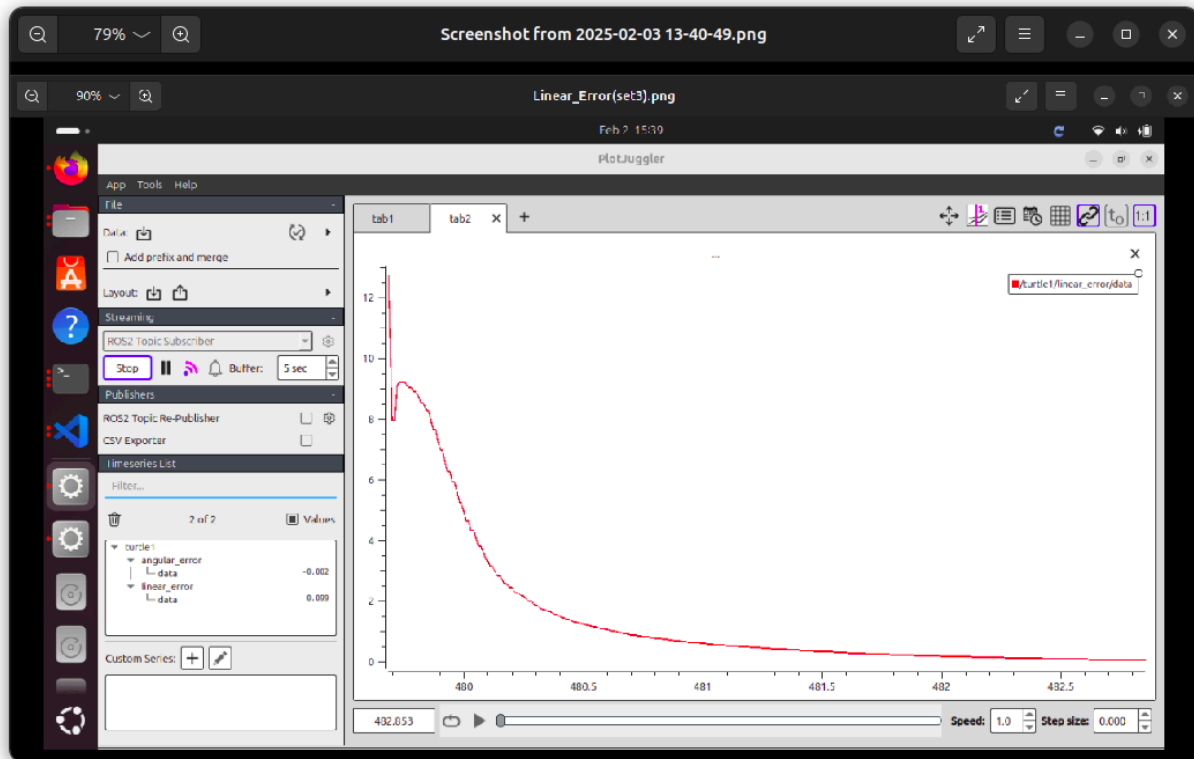
Angular and linear velocity error for gain values (linear: 2.0, 0.0, 0.3 and angular : 5.0, 0.0, 0.5)



Linear and angular velocity error for gain values (linear: 4.0, 0.1, 0.3 and angular : 6.0, 0.3, 0.5)



Linear and linear velocity error for gain values (linear: 3.0, 0.0, 1.0 and angular : 3.0, 0.0, 0.9)



The angular did not change much.

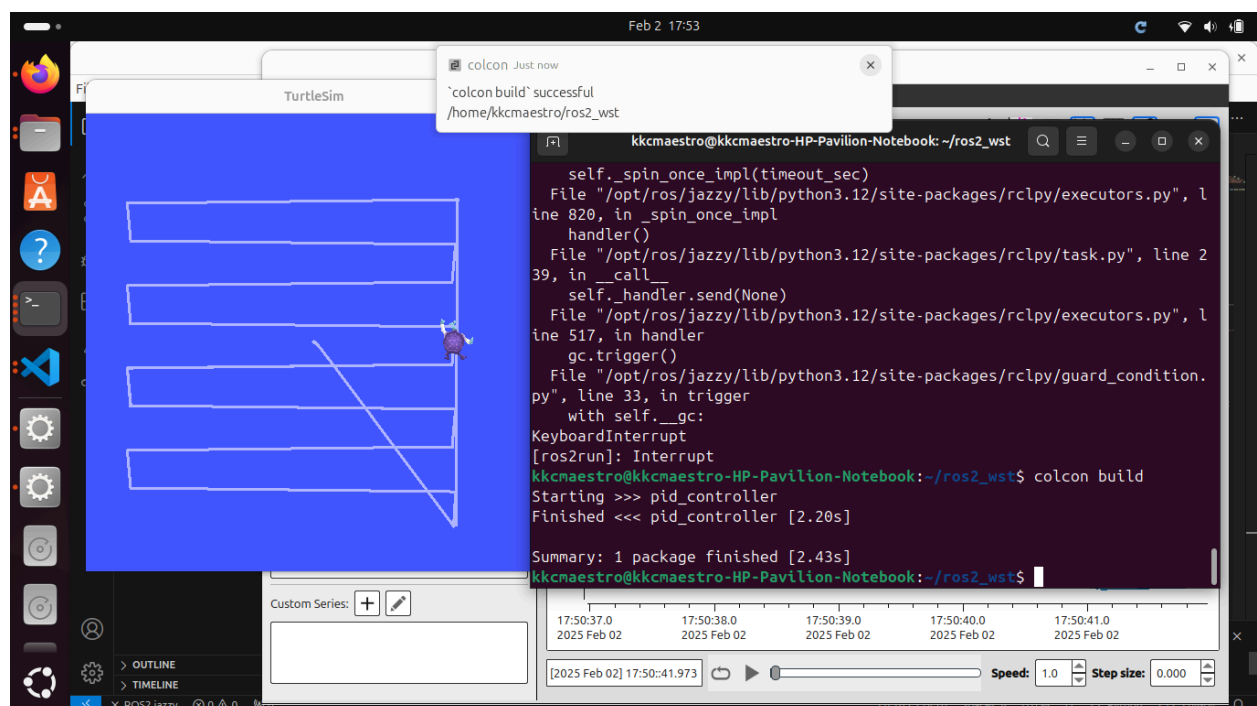
Goal 2: Implementing Inertia and Grid pattern tracing.

To simulate inertia, I created a `deceleration_node` that subscribed to the velocity values from Goal 1. When the primary node stopped publishing velocity values, the `deceleration_node` treated the last received value as the current velocity and applied a gradual deceleration effect by multiplying the velocity value by a number less than 1 and when it reached a threshold it is set to 0. Initially, I considered an exponential loop which is used for controller +ve and -ve acceleration for my stepper motors for this, but the chosen approach was simpler and achieved the same result.

To impose acceleration and deceleration limits, I introduced threshold values. These were integrated into the main motion execution function using conditional statements to regulate velocity changes.

To form a grid using the PID controller, I defined the grid size and created a function to move the turtle to waypoints, rotate, and then proceed to the next goal. Waypoints were stored in an array, representing the grid's extremities (e.g., $(1, 9) \rightarrow (2, 9) \rightarrow (2, 1)$). These waypoints were sequentially assigned as new goals for the turtle.

While tuning the PID gains, I observed that modifying them had minimal impact, except for the integral gain (K_i), where even a small value (e.g., 0.01) caused instability. Further fine-tuning could improve path accuracy, but small adjustments had a significant effect, making it challenging.



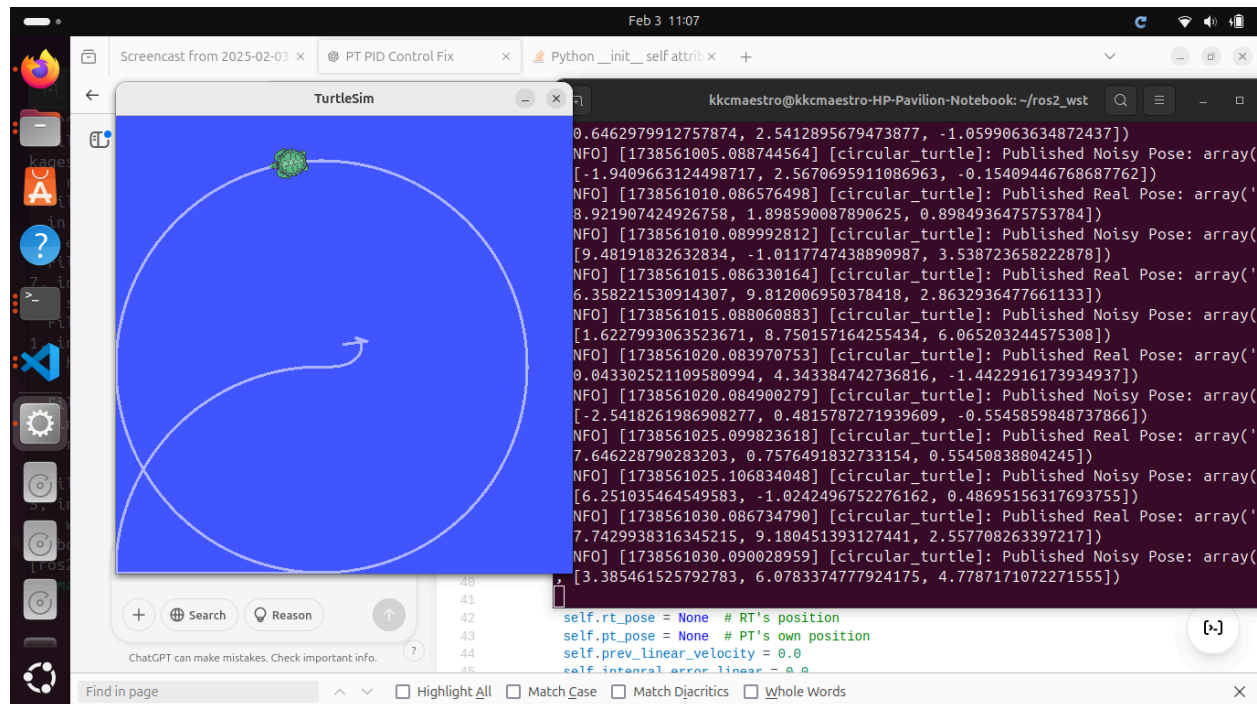
I have shared videos of change in velocity with different max acceleration and erratic graph of acceleration.

Goal 3: Circular Motion and Noisy Data Publishing

For circular motion, a PID controller was unnecessary. Instead, I created a `circle_node` that continuously published velocity commands for circular movement. The radius and velocity were adjustable.

To publish real and noisy position data every 5 seconds, I used `Float64MultiArray`, which supports multiple floating-point values in a single message. A separate timer was set to run at **0.2 Hz**, and a function was created to handle the publishing process.

To introduce random Gaussian noise, I used Python's built-in `random.gauss` function, setting the mean to `0` and the standard deviation to `3`, as specified in the instructions. The node published both real and noisy positions every 5 seconds.



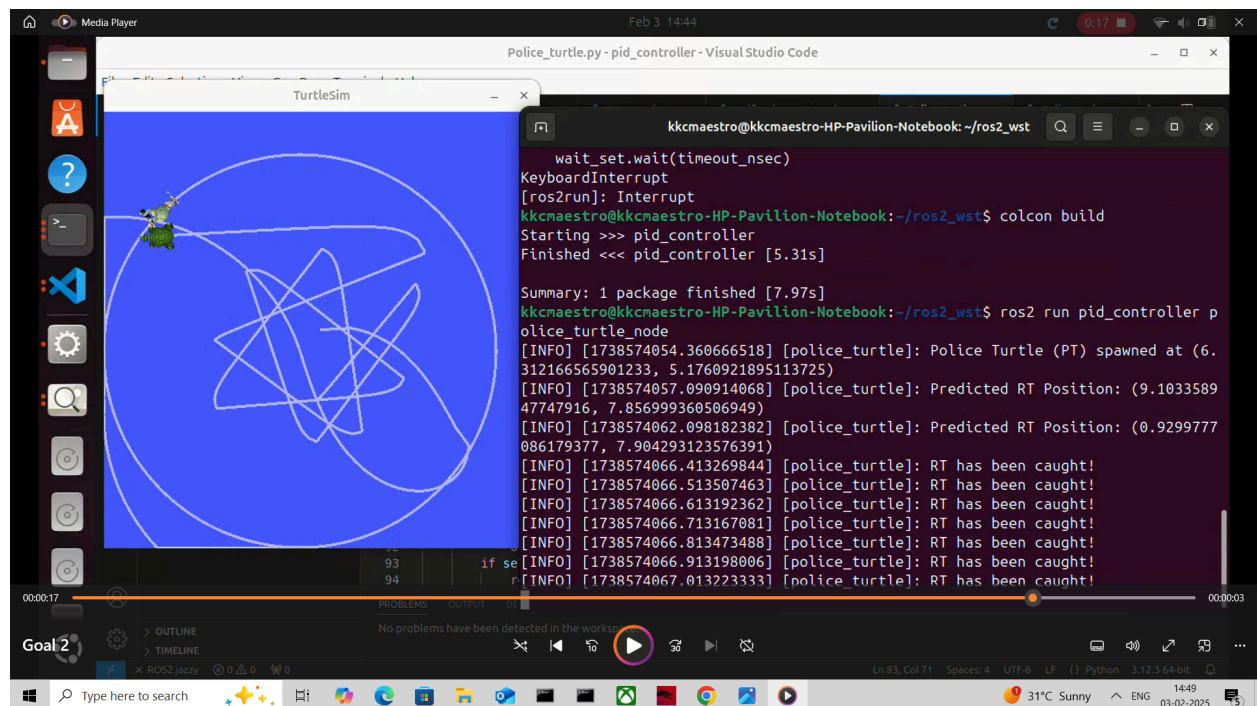
Goal 4: Chasing a Moving Turtle (Robber & Police)

To implement a chase scenario, I used `circle_node` for circular movement and `deceleration_node` to handle gradual stopping. These nodes ran in separate terminals.

For the **Police Turtle**, I created a separate node responsible for spawning, subscribing, and publishing values. Multithreading and executor functions would have been required to combine all turtles in a single script, but I lacked experience in this area.

The **Police Turtle's** movement relied on a PD controller. Since it lacked real-time access to the **Robber Turtle's** position, I implemented a **path predictor**. I assumed the **Robber Turtle** moved in a circular path centered at (5, 5) with a constant speed. Using trigonometric functions (`math.cos` and `math.sin`), I predicted its future position within a 5-second window.

The **Police Turtle** received the initial position, processed it, predicted the target's future location, and moved toward that goal. It ignored subsequent position updates to avoid unnecessary recalculations. If the predicted position was within 0.5 units of the **Robber Turtle**, the police turtle stopped, and the capture was declared.



Goal 5: Extended Prediction for Capture

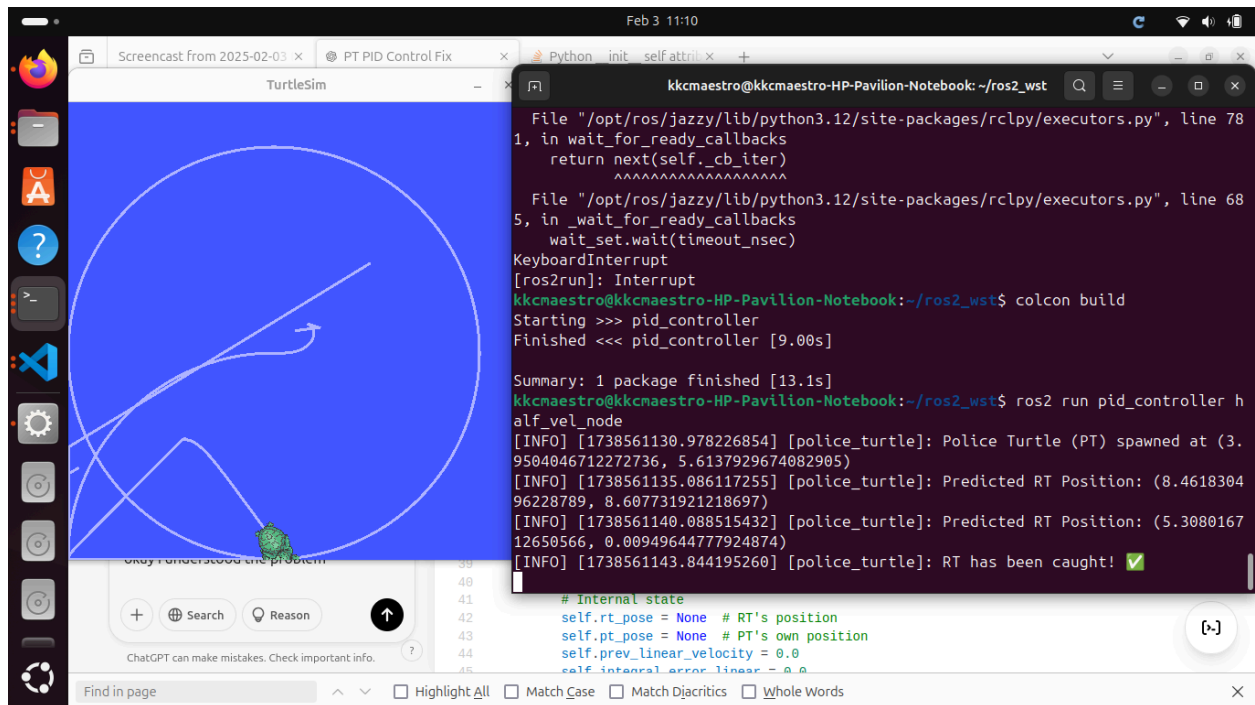
This goal followed the same approach as **Goal 4**, with the only modification being an extended prediction window. The **Robber Turtle's** position was predicted for twice the usual duration by modifying the angle calculation (`math.cos(2*theta)`, `math.sin(2*theta)`).

All previous assumptions from **Goal 4** remained the same.

Goal 6: Filtering Noisy Data Using a Kalman Filter

To process noisy data effectively, I implemented **Kalman Filters**, which are lightweight and efficient for noise reduction. Having prior experience with Kalman Filters, I was able to integrate them seamlessly.

I added two additional processing blocks to compute and apply the filter values, reducing noise. The filtered data was then used to improve the accuracy of the **Police Turtle's** pursuit in **Goals 4 & 5**.



Areas for Improvement:

1. **Acceleration Plot Stability:** The acceleration plot in **Goal 2** was erratic. Smoother transitions could improve realism.
2. **Code Modularity:** The `pid_controller_node` could be refactored for better modularity by separating goal setting, calculations, and publishing into dedicated functions. Attempts to modularize the code led to issues, so I opted for a simpler structure.
3. **Environment Reset Feature:** A reset node could be created to restart the **Turtlesim** environment, allowing for fresh spawn points.
4. **Parameterization of Constants:** Some hardcoded values (e.g., speeds, thresholds) could be converted into configurable parameters for better flexibility.
5. **Dynamic Goal Assignment:** The **Go-to-Goal** program could be improved by dynamically assigning new goals upon reaching the previous one.

6. **Refining the Chaser Algorithm:** The **Police Turtle's** tracking algorithm makes assumptions that may sometimes lead to inaccuracies. Refinements could enhance its reliability.

Conclusion:

This project successfully implemented various motion planning techniques in **Turtlesim**, incorporating PID control, trajectory prediction, Gaussian noise simulation, and Kalman filtering. While the implementation worked as expected, improvements in modularity, parameter tuning, and robustness could enhance overall performance.

All the result and graph video are in a google drive link shared along with this in the email.